

**Sri Sivasubramaniya Nadar College
of Engineering** (An Autonomous Institution affiliated
to Anna University, Chennai)

**Experiment 2: Loan Amount
Prediction** using Linear Regression & SVR

Name	A.S. Tritthik Thilagar
Reg. No.	3122237001057
Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Subject	ICS1512 & Machine Learning Algorithms Laboratory
Academic Year	2025-2026 (Odd)

Contents

1	Aim	2
2	Libraries Used	2
3	Objective	2
4	Mathematical Description	2
5	Code and Results	2
5.1	Linear Regression: Code Implementation	2
5.2	Linear Regression: Output	4
6	Results Tables (Linear Regression)	7
6.1	Cross-Validation Results (K=5)	7
6.2	Summary of Results	7
7	Support Vector Regression (SVR)	8
7.1	SVR: Code Implementation	8
7.2	SVR: Output	8
7.3	Summary of SVR Results	9
8	Best Practices & Learning Outcomes	9
8.1	Best Practices	9
8.2	Learning Outcomes	10

1 Aim

To apply supervised learning regression algorithms, specifically Linear Regression and Support Vector Regression (SVR), to predict the loan amount sanctioned to users based on the provided dataset.

2 Libraries Used

The experiment utilizes the following core Python libraries:

- **Pandas**: For data manipulation and loading CSV files.
- **NumPy**: For numerical operations and data transformation.
- **Matplotlib & Seaborn**: For data visualization, including plotting actual vs. predicted values and correlation heatmaps.
- **Scikit-learn**: For implementing the regression models (Linear Regression, SVR), preprocessing data (pipelines, scaling, encoding), and evaluating performance (cross-validation, R^2 score).

3 Objective

The primary objective is to develop a robust machine learning pipeline in Python to build and evaluate regression models for loan amount prediction. This involves comprehensive data preprocessing, feature engineering, model training using K-Fold Cross-Validation, and hyperparameter tuning for SVR using GridSearchCV.

4 Mathematical Description

Linear Regression is a supervised algorithm that models the linear relationship between a dependent variable (y) and one or more independent variables (X). The model is represented by the equation:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n \quad (1)$$

where y is the predicted value, w_0 is the intercept, and $w_1, \dots, w_n$ are the feature coefficients. The coefficients are found by minimizing a cost function, typically the Mean Squared Error (MSE).

5 Code and Results

5.1 Linear Regression: Code Implementation

The full pipeline for the Linear Regression model is shown below. It covers data loading, preprocessing, outlier treatment, pipeline creation, 5-fold cross-validation, and final model training and prediction.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from google.colab import drive
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import r2_score
8
9 # --- 1. Load Data ---
10 drive.mount('/content/drive')
11 train_path = '/content/drive/MyDrive/Colab Notebooks/EX2/train.csv'
12 test_path = '/content/drive/MyDrive/Colab Notebooks/EX2/test.csv'
13 train_df = pd.read_csv(train_path)
14 test_df = pd.read_csv(test_path)
15
16 print("Loaded the datasets")
17 print("Train shape:", train_df.shape)
18 print("Test shape :", test_df.shape)
19 print("-" * 30)
20
21 # --- 2. Preprocessing and Outlier Treatment ---
22 drop_cols = ['Customer ID', 'Name', 'Property ID', 'Expense Type 1', 'Expense Type 2']
23 train_df.drop(columns=drop_cols, inplace=True, errors='ignore')
24 test_df.drop(columns=drop_cols, inplace=True, errors='ignore')
25
26 # Replace '?' with NaN and convert datatypes
27 for df in [train_df, test_df]:
28     df.replace('?', np.nan, inplace=True)
29     for col in df.columns:
30         df[col] = pd.to_numeric(df[col], errors='ignore')
31     for col in df.select_dtypes(include=['float64', 'int64']):
32         df[col] = df[col].fillna(df[col].mean())
33     for col in df.select_dtypes(include=['object']):
34         df[col] = df[col].fillna(df[col].mode()[0])
35
36 # Fix Co-Applicant corrupted values
37 train_df['Co-Applicant'] = train_df['Co-Applicant'].replace(-999, 0)
38 if 'Co-Applicant' in test_df.columns:
39     test_df['Co-Applicant'] = test_df['Co-Applicant'].replace(-999, 0)
40
41 # Clip outliers
42 outlier_cols = ['Income (USD)', 'Loan Amount Request (USD)', 'Property Price', 'Property Age']
43 for col in outlier_cols:
44     if col in train_df.columns:
45         q_low = train_df[col].quantile(0.01)
46         q_hi = train_df[col].quantile(0.99)
47         train_df[col] = train_df[col].clip(lower=q_low, upper=q_hi)
48     if col in test_df.columns:
49         test_df[col] = test_df[col].clip(lower=q_low, upper=q_hi)
50
51 print("Preprocessing complete.")
52 print("-" * 30)
53
54 # --- 3. Define Features and Target ---
55 X = train_df.drop(columns=['Loan Sanction Amount (USD)'])
56 y = train_df['Loan Sanction Amount (USD)']

```

```

57 x_train,x_test,y_train,y_test=train_test_split(X,y,test_size=0.3,
    random_state=0)
58 X_test_final = test_df # Renamed to avoid confusion
59
60 # --- 4. Create Pipeline using Linear Regression ---
61 from sklearn.compose import ColumnTransformer
62 from sklearn.pipeline import Pipeline
63 from sklearn.preprocessing import StandardScaler, OneHotEncoder
64 from sklearn.linear_model import LinearRegression
65 from sklearn.model_selection import cross_val_score, KFold
66
67 categorical_features = X.select_dtypes(include='object').columns
68 numerical_features = X.select_dtypes(include='number').columns
69
70 preprocessor = ColumnTransformer(
71     transformers=[
72         ('num', StandardScaler(), numerical_features),
73         ('cat', OneHotEncoder(handle_unknown='ignore'),
74         categorical_features)
75     ])
76
77 model_pipeline = Pipeline(steps=[
78     ('preprocessor', preprocessor),
79     ('regressor', LinearRegression())
80 ])
81
82 print("Linear Regression pipeline ready.")
83 print("-" * 30)
84
85 # --- 5. K-Fold Cross-Validation ---
86 print("Performing 5-Fold Cross-Validation with Linear Regression...")
87 kf = KFold(n_splits=5, shuffle=True, random_state=42)
88 scores = cross_val_score(model_pipeline, X, y, cv=kf, scoring='r2')
89
90 print("-" * 30)
91 print(f"Cross-Validation R2 Scores: {np.round(scores, 3)}")
92 print(f"Average R2 Score: {scores.mean():.4f}")
93 print(f"Standard Deviation of R2: {scores.std():.4f}")
94 print("-" * 30)
95
96 # --- 6. Train Final Model & Predict ---
97 print("Training final Linear Regression model...")
98 model_pipeline.fit(x_train, y_train)
99 y_pred=model_pipeline.predict(x_test)
100 r2=r2_score(y_test,y_pred)
101 print("*** R2 SCORE: ",r2)

```

Listing 1: Python code for Linear Regression model.

5.2 Linear Regression: Output

Loaded the datasets

Train shape: (30000, 24)

Test shape : (20000, 23)

Preprocessing complete.

Linear Regression pipeline ready.

Performing 5-Fold Cross-Validation with Linear Regression...

Cross-Validation R2 Scores: [0.627 0.624 0.611 0.622 0.656]

Average R2 Score: 0.6279

Standard Deviation of R2: 0.0149

Training final Linear Regression model...

*** R2 SCORE: 0.6298398990460499

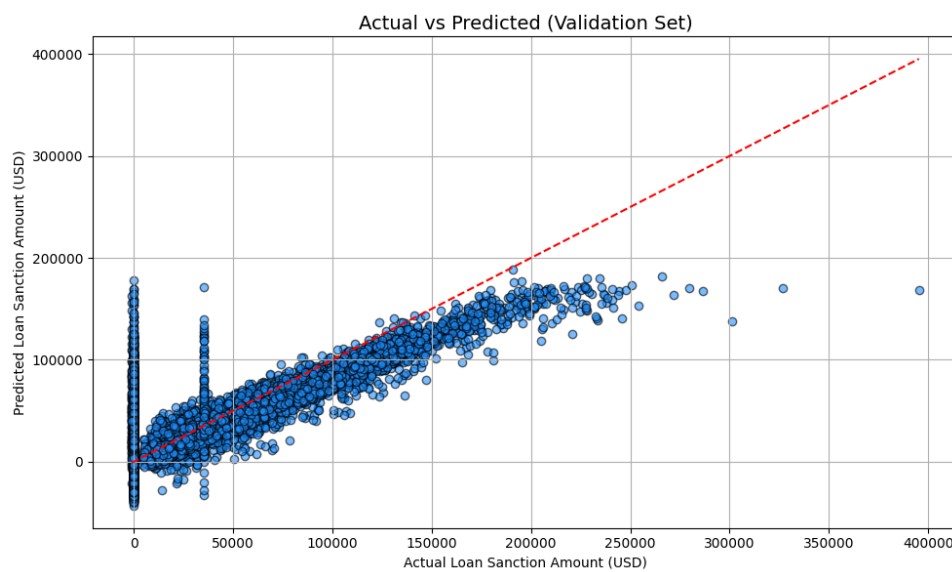


Figure 1: Actual vs. Predicted Loan Sanction Amount (Linear Regression).

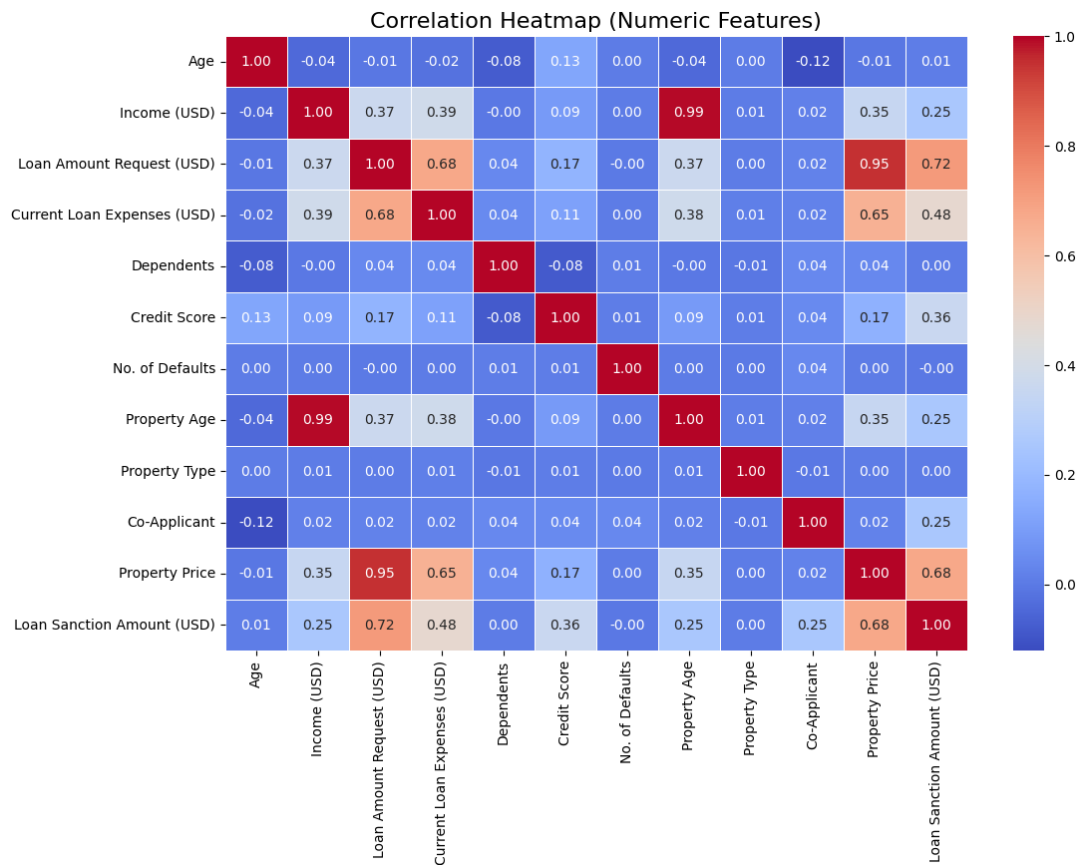


Figure 2: Correlation Heatmap of Numeric Features.

6 Results Tables (Linear Regression)

6.1 Cross-Validation Results (K=5)

Table 1: Cross-Validation R^2 Scores for Linear Regression.

Fold / Metric	R^2 Score
Fold 1	0.627
Fold 2	0.624
Fold 3	0.611
Fold 4	0.622
Fold 5	0.656
Average R^2 Score	0.628
Standard Deviation of R^2	0.015

6.2 Summary of Results

Table 2: Summary of Results for Linear Regression.

Description	Student's Result
Dataset Size (train)	30,000 rows, 24 columns
Train/Test Split Ratio	70% Training / 30% Testing
Model Used	Linear Regression
Cross-Validation Used? (Yes/No)	Yes
Number of Folds (K)	5
Reference to CV Results Table	Table 1
Final R^2 Score on Test Set	0.630
Most Influential Feature(s)	Based on the correlation heatmap, 'Loan Amount Request (USD)' and 'Property Price' have the strongest positive correlations with the target variable.
Interpretation of Predicted vs Actual Plot	The plot shows a clear positive linear relationship. The points are clustered around the diagonal line, indicating that the model's predictions are generally close to the actual values.

7 Support Vector Regression (SVR)

7.1 SVR: Code Implementation

An SVR model was also trained using the same pipeline structure. GridSearchCV was used to find the optimal hyperparameters for 'C', 'epsilon', and 'kernel'.

```

1 # NOTE: This code assumes the data loading and splitting from the
  previous part.
2 # The 'X_train', 'y_train', 'X_test', 'y_test' variables are reused.
3
4 from sklearn.model_selection import GridSearchCV
5 from sklearn.svm import SVR
6
7 # 5. Create the Full Model Pipeline (re-defining for clarity)
8 model_pipeline_svr = Pipeline(steps=[
9     ('preprocessor', preprocessor),
10    ('regressor', SVR())
11 ])
12
13 # 6. Hyperparameter Tuning
14 param_grid = {
15     'regressor__C': [0.1, 1, 10, 100],
16     'regressor__epsilon': [0.01, 0.1, 1],
17     'regressor__kernel': ['linear', 'rbf']
18 }
19
20 grid_search = GridSearchCV(model_pipeline_svr, param_grid, cv=5,
21                             scoring='r2', n_jobs=-1)
22 grid_search.fit(X_train, y_train)
23
24 print("Best hyperparameters from Grid Search:")
25 print(grid_search.best_params_)
26 print(f"Best CV R2 Score from training: {grid_search.best_score_:.4f}")
27
28 # 7. Evaluate the Best Model on the Test Set
29 best_model = grid_search.best_estimator_
30 y_test_pred = best_model.predict(X_test)
31 test_r2 = r2_score(y_test, y_test_pred)
32 print(f"Final R2 Score on the unseen test set: {test_r2:.4f}")

```

Listing 2: Python code for SVR model with GridSearchCV.

7.2 SVR: Output

Best hyperparameters from Grid Search:

```
{'regressor__C': 10, 'regressor__epsilon': 1, 'regressor__kernel': 'linear'}
```

Best CV R2 Score from training: 0.4968

Final R2 Score on the unseen test set: 0.4415

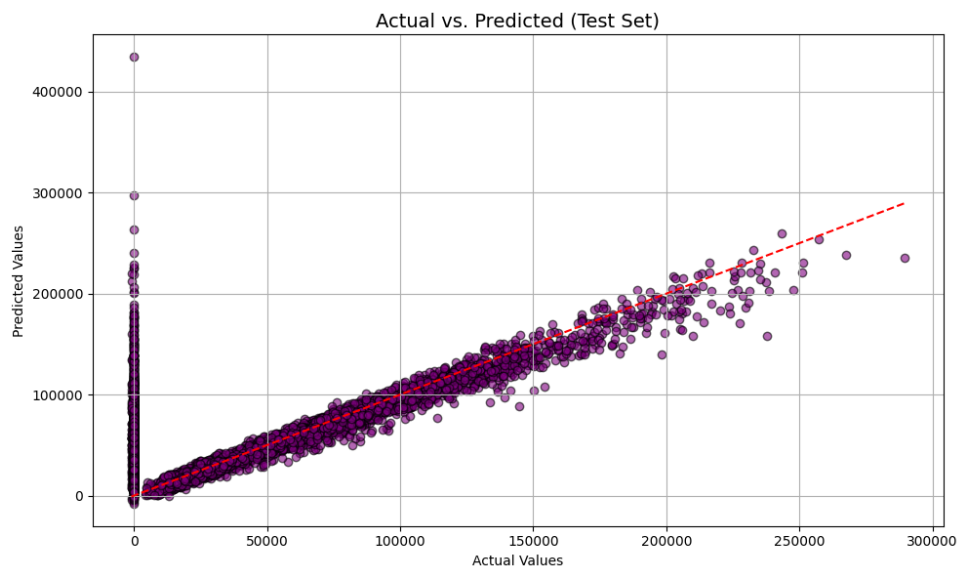


Figure 3: Actual vs. Predicted Loan Sanction Amount (SVR).

7.3 Summary of SVR Results

Table 3: Support Vector Regression Performance Summary.

Metric	Value
Best Kernel	linear
Best C	10
Best Epsilon	1
Best CV R^2 Score (from training)	0.4968
Final R^2 Score on Test Set	0.4415
Observations	The SVR model shows signs of underfitting, as its performance is notably lower than the simpler Linear Regression model. This suggests that a linear kernel SVR might not be complex enough or that different hyperparameter ranges are needed.

8 Best Practices & Learning Outcomes

8.1 Best Practices

- **Using Pipelines:** Encapsulating preprocessing and modeling steps into a pipeline ensures consistency and prevents common errors like data leakage.
- **Cross-Validation:** Using K-Fold Cross-Validation provides a more reliable and robust estimate of the model's generalization performance compared to a single train-test split.

- **Handling Unknown Categories:** Using `handle_unknown='ignore'` in the `OneHotEncoder` makes the model robust to new categories that might appear in future test data.

8.2 Learning Outcomes

Through this experiment, I learned to:

- Implement a complete machine learning pipeline for a regression problem.
- Systematically preprocess data using Scikit-learn's `ColumnTransformer`.
- Build, train, and evaluate both Linear Regression and SVR models.
- Understand and apply cross-validation for robust model assessment.
- Perform hyperparameter tuning using `GridSearchCV` to find the optimal SVR model.
- Interpret and compare the performance of different regression models using the R^2 score.